Time Tracker Chrome Extension — Project Documentation

A complete description of the project: purpose, features, design, architecture and usage.

1. Project overview

Time Tracker is a lightweight Chrome extension that lets a person record what they work on during the day, see how their time is divided, and export the result to CSV or Excel. It runs entirely inside the browser. There is no account, no server and no network traffic. All data is stored locally using Chrome's extension storage.

The extension sits in the browser toolbar. Clicking its icon opens a popup where the user can start a timer for a task, pause or resume it, stop it, review today's sessions and download a report.

1.1 Problem statement

People who work on several tasks in a day, such as freelancers, students, developers and office workers, often lose track of where their time goes. Many existing tools are heavy, require sign-ups, keep data in the cloud, or are designed for teams rather than individuals. A small, private, always-available tracker that lives in the browser and produces a clean daily report solves this for personal use.

1.2 Objectives

- Make starting and stopping a timer a one-click action.
- Keep timing accurate even when the popup is closed.
- Give a clear daily summary at a glance.
- Produce reports that open correctly in Excel and Google Sheets.
- Keep all data private and local.
- Stay dependency-free, with no frameworks and no build step, so the code is easy to read, maintain and publish.

1.3 Target users

- Freelancers who bill by the hour.

- Students tracking study sessions.
- Developers and designers logging project work.
- Anyone who wants a simple daily time report without a cloud service.

2. Features

2.1 Timer controls

- **Start** a session with a title, category, status and optional notes.
- **Pause and resume.** Paused time is never counted toward the session.
- **Stop** to save the session to today's log.
- Only one session can run at a time. Trying to start a second one is blocked with a clear message.

2.2 Task details

- **Categories:** Work, Study, Development, Meeting, Break, Lunch, Personal, Other.
- **Statuses:** In Progress, Not Started, Completed, On Hold.
- **Notes:** free text, optional.

2.3 Today's summary

- Total tracked time for the day, including the running session.
- Session count and today's date.
- A horizontal bar and legend showing how today's time splits across the four statuses.
- A header badge showing Idle, Tracking or Paused.

2.4 Today's time logs

Each saved session is shown as a card with its title, duration, category, start and end times, status and notes. The card colour is driven by status only:

Status	Colour
In Progress	Blue
Completed	Green

Status	Colour
On Hold	Amber
Not Started	Slate grey

The left stripe, the duration pill and the status chip on a card always share the same colour. Each card has a soft shadow that lifts slightly on hover, a **Change status** button and a **Delete** button.

2.5 Export

- **CSV** export, opened cleanly by Excel and Google Sheets.
- **Excel (.xlsx)** export with auto-fitted column widths, a styled header row, a frozen header and filters.
- A running session, if any, is included in both formats and marked Running or Paused.

2.6 Feedback and confirmations

- A **custom confirmation dialog** for deleting a log and for clearing the day, instead of the browser's built-in confirm box.
- **Toast notifications** for every message. One small notice appears at the bottom of the popup, stays visible wherever the user has scrolled, and dismisses itself.

2.7 Toolbar badge

A green dot appears on the extension icon while a task is running and an amber "II" while it is paused. The badge is cleared when no session exists.

2.8 Interface details

- A thin 4 px scrollbar.
- A custom stopwatch logo in every size Chrome requires.
- A consistent colour system based on the logo's blue-to-teal gradient.

3. Installation and usage

3.1 Installing

1. Download or clone the repository.

2. Open `chrome://extensions` in Chrome (or Edge, Brave or Opera).
3. Turn on **Developer mode**.
4. Click **Load unpacked** and choose the folder that directly contains `manifest.json`.
5. Pin the extension from the puzzle-piece menu.

After editing the code, click the reload icon on the extension's card and reopen the popup.

3.2 Daily workflow

1. Enter a task title, choose a category and status, optionally add notes, and press **Start tracking**.
2. Use **Pause** and **Resume** during breaks or interruptions.
3. Press **Stop session** when finished. The session appears in today's logs.
4. Adjust a log's status with **Change status**, or remove it with **Delete** after confirming.
5. Press **CSV** or **Excel** to download the day's report.
6. Use **Clear today's logs** to wipe the day after confirming. A running session must be stopped first.

4. Technology stack

Layer	Choice	Reason
Platform	Chrome Extension, Manifest V3	Current standard, required for new store submissions
Markup	HTML5	A single popup page
Styling	Plain CSS with custom properties	No build step, themeable through variables
Logic	Vanilla JavaScript	No dependencies, small and easy to audit
Storage	`chrome.storage.local`	Persistent, private, available to popup and service worker
Background	Manifest V3 service worker	Keeps the toolbar badge in sync

Layer	Choice	Reason
Export	Hand-written CSV and XLSX writers	No third-party libraries
Icons	PNG at four sizes plus an SVG source	Logo at 16, 32, 48 and 128 px

No packages are installed and nothing needs to be compiled.

5. Architecture

5.1 Components

- **Popup** (`popup.html` , `popup.css` , `popup.js`): the entire user interface and application logic.
- **Service worker** (`background.js`): listens for storage changes and updates the toolbar badge. It contains no business logic.
- **Storage:** a single record in `chrome.storage.local` , shared by both components.

The popup is created each time the icon is clicked and destroyed when it closes. For this reason the timer never relies on the popup staying open, as explained in section 6.1.

5.2 Data flow

1. The popup loads saved data from storage and renders it.
2. A user action (start, pause, stop, change status, delete, clear) updates the in-memory state object.
3. The popup saves the state to storage.
4. The popup re-renders. Separately, the service worker receives the storage change and updates the badge.
5. A one-second interval re-renders the live timer, and a storage change listener refreshes the view if the data is updated elsewhere.

5.3 Data model

All data is stored under the key `timeTrackerData` :

```
{
  "logs": [
    {
      "id": "uuid",
      "title": "Build my portfolio",
      "category": "Development",
      "status": "Completed",
      "notes": "Landing page",
      "startAt": 1761370000000,
      "endAt": 1761372700000,
      "durationMs": 2700000
    }
  ],
  "active": {
    "id": "uuid",
    "title": "Write API docs",
    "category": "Work",
    "status": "In Progress",
    "notes": "",
    "startAt": 1761373000000,
    "resumedAt": 1761373000000,
    "accumulatedMs": 0,
    "state": "running"
  }
}
```

`active` is `null` when nothing is being tracked. `state` is either `running` or `paused`. All times are millisecond timestamps. The calendar day of a log is derived from `startAt` in the user's local time zone.

6. Key design decisions

6.1 Timestamp-based timing

The timer does not count ticks. A running session stores when it last resumed and how much time it had already accumulated. Elapsed time is calculated on demand:

`elapsed = accumulatedMs + (now − resumedAt)` while running, and `accumulatedMs` while paused.

Pausing folds the elapsed time into `accumulatedMs` and clears `resumedAt`. The result stays accurate when the popup is closed, when the computer sleeps, and after a browser restart. Pause time is excluded by construction.

6.2 Local day boundaries

The day key is built from the browser's local date (year, month, day), not from UTC. A session started at 11:50 pm belongs to the date the user sees on their clock. A session is assigned to the day on which it started.

6.3 Rendering strategy

The popup re-renders once per second to update the timer. Rebuilding the list of log cards every second would interrupt hover effects and could swallow clicks, so the list keeps a signature of its data and is rebuilt only when that signature changes. The timer, summary and split bar are small and update every second.

6.4 Status-driven colour

The interface uses a `data-status` attribute that sets two CSS variables: a strong colour and a light tint. The log card stripe, the duration pill, the status chip, the active-session card and the summary legend all read the same variables. A status colour therefore means the same thing everywhere.

6.5 Custom dialog and toast instead of browser alerts

Browser `alert` and `confirm` boxes block the page and look out of place. The extension uses a promise-based `confirmDialog()` that resolves true or false, supports Escape and click-outside to cancel, and focuses the safe Cancel button first. Messages use one shared `showMessage(text, isError)` function that drives a single fixed-position toast, so every section reports feedback in the same place and the user never has to scroll to see it. Success toasts last about 2.4 seconds and errors about 3.8 seconds. Both can be dismissed by clicking.

7. Export implementation

7.1 Columns

Both formats contain the same columns: Task Title, Category, Date, Start Time, End Time, Duration (HH:MM:SS), Duration (Minutes), Task Status, Session State, Notes. Rows are sorted oldest first. Session State is Stopped, Running or Paused.

7.2 CSV

The CSV file is UTF-8 with a byte-order mark so Excel recognises special characters, uses Windows line endings, and wraps every field in quotes. The Date, Start Time, End Time and Duration (HH:MM:SS) columns are written as `="value"`, which makes Excel treat them as text. This keeps dates and times shown exactly as they are formatted instead of being converted to numbers.

A CSV file cannot store column widths, so a column may look narrow when first opened and can simply be widened. For a workbook that is ready to read immediately, use the Excel export.

7.3 Excel (.xlsx)

An `.xlsx` file is a ZIP archive of XML parts. The extension builds these parts as strings:

- `[Content_Types].xml` and `_rels/.rels` describe the package.
- `xl/workbook.xml` and its relationships define one sheet named "Time Log".
- `xl/styles.xml` defines a bold white header on a blue fill and plain text for data.
- `xl/worksheets/sheet1.xml` holds the rows, the column widths, a frozen top row and an auto-filter.

Column widths are calculated from the longest value in each column, with a minimum of 10 and a maximum of 60 characters. The parts are packed by a small ZIP writer that stores files uncompressed and computes each file's CRC-32 checksum with a lookup table. All cells are written as inline strings, which avoids a shared-strings table. Control characters are removed and XML special characters are escaped.

7.4 Spreadsheet injection protection

Text that begins with `=`, `+`, `-` or `@` can be interpreted as a formula when opened in a spreadsheet. Such values are prefixed with an apostrophe in both formats, so user-entered titles and notes cannot execute as formulas.

8. User interface design

8.1 Layout

The popup is 390 px wide with a maximum height of 600 px and a thin internal scrollbar. From top to bottom it contains:

1. **Header:** logo, product name, tagline and a state badge.
2. **Summary card:** a dark gradient card with today's total in large type, the date, the status split bar and legend, and the session count.
3. **Current session:** an empty state, or the running task in a card tinted with its status colour, with a pulsing indicator, a live timer, task details, and Pause and Stop buttons.
4. **Start a new task:** the entry form.
5. **Today's time logs:** the CSV and Excel buttons and the list of session cards.
6. **Footer:** the Clear today's logs button.

The toast and the confirmation dialog overlay the whole popup.

8.2 Design system

- **Brand:** deep ink navy with a blue-to-teal gradient taken from the logo.
- **Typography:** the system font stack, with tabular numerals for every timer so digits do not jitter.
- **Shape:** rounded corners from 10 to 18 px, soft layered shadows and tinted chips.
- **Semantic colour:** blue, green, amber and slate for status; red for destructive actions; amber for pause.
- **Motion:** a pulse on the live indicator, a short pop-in for dialogs and toasts, and a small lift when hovering a log card. All are CSS-only.

8.3 Accessibility

The toast uses `role="status"` so it is announced politely, and the confirmation dialog uses `role="alertdialog"` with a labelled title and description. Form fields have visible labels, controls are real buttons that can be reached with the keyboard, and inputs show a clear focus ring.

9. File reference

File	Purpose
`manifest.json`	Extension metadata, permissions, icons, popup and service worker declarations
`background.js`	Updates the toolbar badge from storage changes
`popup.html`	Popup markup, toast element and confirmation dialog
`popup.css`	All styling, variables, animations and the thin scrollbar
`popup.js`	State, timer maths, storage, rendering, dialogs, toast, CSV and XLSX export
`icons/`	The logo at 16, 32, 48 and 128 px, and an SVG source
`screenshots/`	Preview image used by the README
`README.md`	Public repository overview
`LICENSE`	MIT licence
`.gitignore`	Excludes OS files, editors, archives and dependencies

Main functions in `popup.js` include `load`, `save`, `startTask`, `togglePause`, `stopTask`, `changeStatus`, `deleteLog`, `clearToday`, `render`, `renderActive`, `renderSplit`, `renderLogs`, `confirmDialog`, `showMessage`, `exportCsv`, `exportXlsx`, `buildXlsx` and `zipStore`.

10. Security and privacy

- **Permissions:** only `storage`. There are no host permissions, no access to tabs and no access to browsing history.
- **Network:** the extension makes no network requests and loads no remote code, fonts or scripts.

- **Output safety:** every user-supplied string is HTML-escaped before it is inserted into the page, which prevents script injection through titles or notes. Export values are protected against formula injection.
- **Storage:** data stays on the device in extension storage and is not synced. Removing the extension deletes it, so users should export first if they want to keep a copy.

11. Manual test checklist

Before publishing, run through these scenarios in Chrome:

Scenario	Expected result
Start, pause, resume, stop	Time excludes paused periods; the log is saved
Close and reopen the popup while running	The timer continues from the correct value
Restart the browser while running	The session is restored and keeps counting
Delete a log	A dialog appears; the log is removed only after confirming
Clear today's logs while a session runs	Blocked with an error toast
Export CSV and Excel with and without a running session	Files download with the correct columns
Titles beginning with `=` or `+`	Shown as plain text in the exported file
Toolbar badge	Green dot when running, amber when paused, empty when idle

12. Limitations

- Only today's sessions are shown and exported. Older logs remain in storage but have no screen yet.
- One active timer at a time.
- Saved logs cannot be edited, apart from their status.
- A session that crosses midnight is counted entirely on the day it started.
- Data is local to one browser profile and is not synced between devices.

- The popup closes when it loses focus, which is normal Chrome behaviour. The timer is unaffected.

13. Future scope

- A history view and date-range export.
- Editing saved logs, including times and notes.
- Weekly and monthly reports with charts.
- Idle detection and reminders.
- Dark mode.
- Import from CSV, plus backup and restore.
- Optional sync across devices.
- Publication to the Chrome Web Store.

14. Publishing the project on GitHub

1. Replace `YOUR NAME` in `LICENSE` and `<your-username>` in `README.md`.
2. Create an empty repository on GitHub without a README, licence or .gitignore.
3. From the project folder run:

```
git init
git add .
git commit -m "Initial commit: Time Tracker"
git branch -M main
git remote add origin https://github.com/<your-username>/time-tracker-
extension.git
git push -u origin main
```

4. Add repository topics such as `chrome-extension`, `time-tracker`, `productivity` and `manifest-v3`.
5. Optionally create a release and attach a zip of the project.

To publish on the Chrome Web Store, zip the folder contents with `manifest.json` at the top level and upload the archive in the developer dashboard, which requires a one-time registration fee.